

Operations & Monitoring Guide — Family Music Scrobbler PWA

Operations & Monitoring Guide - Family Music Scrobbler PWA

A complete operational reference for monitoring, maintaining, and ensuring the long-term health of the Family Music Scrobbler PWA. This guide covers runtime observability, ingestion worker monitoring, log analysis, backup verification, and production health checks.

1. Overview

This guide is designed for day-to-day and long-term operations of the Family Music Scrobbler PWA running on the Intel NUC (Fedora Server). It ensures:

- Continuous ingestion of Spotify listening history
 - Reliable PWA performance
 - Verified backups
 - Early detection of failures
 - Predictable recovery procedures
-

2. Operational Responsibilities

Daily

- Check /healthz endpoint
- Review ingestion worker logs
- Confirm recent scrobbles are appearing in the dashboard

Weekly

- Verify backup integrity
- Inspect disk usage
- Review Docker container health

Monthly

- Confirm new scrobble partitions are created

- Archive old partitions if desired

Quarterly

- Update system packages
 - Review security posture
-



3. Health Monitoring

3.1 Health Check Endpoint

The backend exposes:

```
GET /healthz
```

This returns:

```
{
  "database": "ok",
  "worker": "running",
  "api": "ok"
}
```

What to look for

- database: ok → PostgreSQL reachable
- worker: running → APScheduler active
- api: ok → FastAPI responding normally

If any field is not ok, investigate immediately.



4. Ingestion Worker Monitoring

4.1 Check Worker Logs

```
docker logs backend --tail 50
```

Look for:

- Polling events every 5-10 minutes
- Successful Spotify API calls
- Deduplication messages
- Backoff events (429/5xx)

4.2 Check Last Polled Timestamp

```
GET /ingestion/status
```

Example:

```
{
  "last_polled": "2026-09-09T07:30:00Z",
  "poll_interval_minutes": 10,
  "status": "running"
}
```

If last_polled is older than 20 minutes, ingestion may be stalled.

5. Database Monitoring

5.1 Check Partition Creation

Monthly partitions should exist:

```
scrobbles_2026_09
scrobbles_2026_10
scrobbles_2026_11
```

To list partitions:

```
docker exec postgres psql -U postgres -c "\d+ scrobbles"
```

5.2 Check Table Size

```
docker exec postgres psql -U postgres -c "SELECT
pg_size_pretty(pg_total_relation_size('scrobbles'));"
```

5.3 Check Index Health

```
docker exec postgres psql -U postgres -c "SELECT * FROM pg_stat_user_indexes;"
```

6. Docker Monitoring

6.1 Check Running Containers

```
docker ps
```

6.2 Check Container Resource Usage

```
docker stats
```

Look for:

- High CPU usage on backend (worker stuck)
- High memory usage on PostgreSQL

6.3 Restart Services

```
docker compose -f infra/docker-compose.yml restart
```

7. Log Management

7.1 Backend Logs

```
docker logs backend
```

7.2 API Request Logs

Stored in structured JSON format.

7.3 Worker Logs

Look for:

- Polling cycles
- Token refresh events
- Backoff retries

7.4 Error Logs

```
docker logs backend | grep ERROR
```

8. Security Monitoring

8.1 Check Secrets Mounting

```
ls /opt/family-scrobblor/secrets/
```

Should contain:

- refresh_token_key
- vapid_private_key
- vapid_public_key

8.2 Check Token Encryption

Verify encrypted tokens exist:

```
docker exec postgres psql -U postgres -c "SELECT refresh_token_enc FROM spotify_tokens  
LIMIT 5;"
```

8.3 Review Authentication Logs

Look for suspicious login attempts.

9. Backup Verification

Backups are validated using:

```
scripts/verify_backup.sh
```

This script:

1. Runs pg_dump
2. Syncs dump to Synology DS223
3. Spins up a temporary PostgreSQL container
4. Restores the dump
5. Runs integrity checks

9.1 Manual Execution

```
cd /opt/family-scrobber/scripts  
sudo bash verify_backup.sh
```

9.2 Check Backup Logs

```
cat /opt/family-scrobber/logs/backup.log
```



10. Alerting & Notifications

10.1 Push Notifications (Frontend)

Used for:

- Leaderboard updates
- Ingestion alerts

10.2 Manual Alerts

You may configure:

- Email alerts for ingestion failures
 - NAS alerts for backup failures
-



11. Troubleshooting

11.1 Ingestion Stopped

Check:

- Worker logs
- Spotify API rate limits
- Token refresh failures
- last_polled timestamp

11.2 Backend Not Responding

Check:

```
docker logs backend
systemctl status docker
```

11.3 Database Errors

Check:

```
docker logs postgres
```

11.4 PWA Offline Issues

Check:

- Service Worker status
 - IndexedDB sync queue
 - Network-first API caching
-

12. Summary

This guide provides everything needed to:

- Monitor ingestion
- Validate system health
- Inspect logs
- Verify backups
- Maintain secure operations

It ensures the Family Music Scrobbler PWA runs reliably in your home-lab environment.